AD Aflevering 3

kvb125, fps618

Mar 2024

1 Amortiseret analyse

En potentialfunktion, Φ, i amortiseret analyse er en funktion designet til at måle systemets "energi" eller "potentiale" på et givet tidspunkt. Denne funktion hjælper med at analysere den samlede køretid af en sekvens af operationer ved at tage højde for variationen i individuelle operationers køretid.

I tilfældet med dynamiske tabeller, som beskrevet i CLRS sektion 16.4.1, bruges potentialfunktionen til at analysere omkostningerne ved at indsætte elementer i en tabel, der udvider sig dynamisk. Når en tabel er fuld og et nyt element skal indsættes, fordobles tabellens størrelse, og alle eksisterende elementer kopieres over til den nye, større tabel. Dette kan synes dyrt, men amortiseret analyse viser, at omkostningen fordelt over alle operationer stadig er konstant, $O(1)$.

Baseret på beskrivelsen og definitionen af potentialfunktionen Φ_i efter i TABLE-INSERT operationer, uden nogen TABLE-DELETE operationer, kan vi vurdere de angivne udsagn:
documentclassarticle

1. $\Phi_i = \Omega(1)$

Dette udsagn er korrekt. Potentialfunktionen defineres som forskellen mellem den aktuelle omkostning og den amortiserede omkostning. Selv når tabellen er tom, er der en basisomkostning forbundet med oprettelsen af tabellen. Dette garanterer, at Φ_i altid vil være større end eller lig med en konstant, fordi potentialfunktionen afspejler den "lagrede energi" eller kapacitet til at absorbere fremtidige omkostninger, der starter fra en grundlæggende ikke-negativ værdi.

2. $\Phi_i = O(1)$

Dette udsagn er ikke korrekt. Den grundlæggende fejl i denne antagelse er, at potentialfunktionens værdi ikke forbliver begrænset af en konstant uafhængigt af antallet af indsættelser. Ved hver tabeludvidelse, som typisk sker ved fordobling af tabellens størrelse, stiger potentialfunktionens værdi for at afspejle de større omkostninger ved fremtidige kopieringsoperationer. Derfor vokser Φ_i med antallet af indsættelser, hvilket gør udsagnet inkorrekt.

3. $\Phi_i = \Omega(i)$

Dette udsagn er ikke nødvendigvis korrekt, da potentialfunktionen ikke nødvendigvis vokser lineært med antallet af operationer i. Væksten af Φ_i er knyttet til, hvordan tabellen udvides dynamisk, hvilket ikke altid er direkte proportionalt med i på grund af fordoblingsstrategien, der anvendes til at øge tabellens størrelse.

4. $\Phi_i = O(i)$

Dette udsagn er korrekt i den forstand, at den øvre grænse for potentialfunktionens vækst er lineær i forhold til antallet af indsættelser. Den amortiserede analyse viser, at selvom omkostningerne ved en enkelt udvidelsesoperation kan være høj, fordeler den amortiserede omkostning per operation sig jævnt over de mange operationer, der førte til udvidelsen, hvilket holder den samlede vækst i Φ_i inden for en lineær øvre grænse.

5. $\Phi_i \geq \Phi_{i-1}$

Dette udsagn er korrekt. Da hver TABLE-INSERT operation potentielt kan øge størrelsen af tabellen og dermed "energien" eller "potentialet" til at absorbere fremtidige omkostninger, vil potentialfunktionens værdi enten vokse eller forblive den samme efter hver indsættelse. Det er derfor, Φ_i altid vil være større end eller lig med Φ_{i-1}.

6. $\Phi_i \geq 0$

Dette udsagn er korrekt. Potentialfunktionen er designet til at måle den "lagrede" eller "forskudte" omkostning og kan der

2 Dynamisk programmering

For at forstå hvad MAXIMUMMYSTERY-proceduren beregner, analyserer vi først pseudokoden:

- Der oprettes et nyt array r[1 ... n], hvor r[j] repræsenterer længden af den længste ikke-aftagende delsekvens, der slutter med elementet A[j].

- Initialiseringsfasen sætter r[1]=1, hvilket angiver, at den længste ikke-aftagende delsekvens, der slutter med det første element, har længden 1.

- For hvert element A[j] fra det andet element til det sidste, gennemgås elementerne A[i] før A[j] (hvor i < j), og der findes den maksimale værdi af r[i]+1 for alle i, hvor A[j] ikke er mindre end A[i]. Dette sikrer, at hvis A[j] kan tilføjes til en delsekvens, der slutter med A[i] uden at bryde reglen om ikke-aftagende orden, opdateres r[j] til at afspejle længden af denne udvidede delsekvens.

- Til sidst returneres den maksimale værdi fundet i arrayet r, hvilket repræsenterer længden af den længste ikke-aftagende delsekvens i hele arrayet A.

Ud fra denne analyse kan vi konkludere, at MAXIMUMMYSTERY-proceduren beregner længden af en længste ikke-aftagende delsekvens i arrayet A. Dette er fordi proceduren tillader, at elementerne A[j] og A[i] er lige (A[j] behøver ikke være strengt større end A[i]), hvilket er kriteriet for en ikke-aftagende delsekvens.

Invariant for den ydre løkke: Ved starten af hver iteration j af den ydre løkke, indeholder r[1 ... j-1] længden af den længste ikke-aftagende delsekvens, der slutter med hvert element A[i], for alle 1 <= i < j. Dette sikrer, at når vi opdaterer r[j], har vi allerede beregnet og gemt længden af de længste ikke-aftagende delsekvenser, der slutter med alle forudgående elementer, hvilket gør det muligt at udlede r[j] korrekt.

Så det korrekte svar er: **4. Længden af en længste ikke-aftagende delsekvens.**

3 Længste fælles delsekvens

Vi opstiller tabellen c som algoritmen udfylder for input $x = \langle N, A, A, N \rangle$ og $y = \langle A, N, N, A \rangle$, og tegner pile ($\uparrow, \nwarrow, \leftarrow$) der forklarer tallene i tabellen.

Tabellen c er som følger, med de tilsvarende pile angivet i hver celle:

		A	N	N	A
	0	0	0	0	0
N	0	0	1	1	1
A	0	1	1	1	2
A	0	1	1	1	2
N	0	1	2	2	2

De tilsvarende pile for hver celle (bortset fra initialiseringens rækker og kolonner) er:

- $\uparrow$ betyder, at værdien blev kopieret fra cellen over.
- $\nwarrow$ betyder, at der var et match, og værdien blev taget fra cellen diagonalt op til venstre plus et.
- $\leftarrow$ betyder, at værdien blev kopieret fra cellen til venstre.

	A	N	N	A
N	↑	↖	←	←
A	↖	↑	↑	↖
A	↑	↑	↑	↖
N	↑	↖	←	←

Fra denne tabel og pile kan vi bestemme en længste fælles delsekvens. Den maksimale værdi i c tabellen er 2 , hvilket indikerer længden af den længste fælles delsekvens. Ved at følge pile tilbage fra cellen $c[4,4]$ kan vi konstruere delsekvensen. For eksempel, hvis vi følger pilene $\nwarrow$ fra $c[4,4]$ til $c[2,1]$, får vi elementerne "A" og "N" som en mulig længste fælles delsekvens, hvilket er "AN".

Så en længste fælles delsekvens mellem $x = \langle N, A, A, N\rangle$ og $y = \langle A, N, N, A\rangle$ er "AN".p

4 Vægtet Længste Fælles Delsekvens

Vi betragter en variant af problemet med længste fælles delsekvens, hvor målet er at maksimere vægten af tegnene i delsekvensen frem for længden.

a) Rekursionsligning

Lad $C[i,j]$ være den maksimale vægt af en fælles delsekvens af præfikserne $x[1..i]$ og $y[1..j]$. Så kan rekursionsligningen for $C[i,j]$ defineres som:

$$C[i,j] = \begin{cases} 0 & \text{hvis } i = 0 \text{ eller } j = 0, \\ C[i-1,j-1] + w[x_i] & \text{hvis } i,j > 0 \text{ og } x_i = y_j, \\ \max(C[i-1,j], C[i,j-1]) & \text{hvis } i,j > 0 \text{ og } x_i \neq y_j. \end{cases}$$

Køretidsanalyse af den vægtede længste fælles delsekvens

For at løse problemet med at finde den vægtede længste fælles delsekvens mellem to sekvenser x og y anvendes en tilgang med dynamisk programmering, der ligner den klassiske metode til at finde den længste fælles delsekvens (LCS). Denne tilgang kræver konstruktionen af en tabel C, hvor hver celle $C[i,j]$ repræsenterer den maksimale vægt af en fælles delsekvens mellem de første i elementer af x og de første j elementer af y.

b) Køretidsanalyse

Algoritmen bygger på følgende rekursive egenskaber:

- Hvis $x_i = y_j$, tilføjes vægten af dette matchende element til den samlede vægt af den bedste delsekvens fundet indtil nu.
- Hvis $x_i \neq y_j$, bestemmes den største vægt ved at sammenligne vægten af delsekvenserne opnået ved enten at ekskludere x_i eller y_j.

Denne metode sikrer, at alle mulige delsekvenser undersøges, og den med den højeste samlede vægt vælges.

Da tabel C har en størrelse på $(m+1) \times (n+1)$ og hver celle i tabellen beregnes i konstant tid, afhænger den totale køretid direkte af tabelstørrelsen. Dette resulterer i en køretid på $O(mn)$, hvilket er analogt til den klassiske LCS-algoritme. Den væsentlige forskel ligger i, hvordan vægten af hver matchende delsekvens bidrager til den samlede beregning, hvilket ikke ændrer den overordnede køretidskompleksitet.

Denne tilgang viser, at selvom problemet er udvidet til at inkludere vægte, forbliver den grundlæggende struktur og dermed køretiden af algoritmen uændret fra den, vi kender fra den klassiske LCS-problemstilling.

c) Håndtering af Negative Vægte

Tilpasningen af algoritmen til at inkludere behandlingen af negative vægte kræver en nøje overvejelse af, hvordan disse vægte indvirker på identifikationen af en optimal delsekvens. Et centralt punkt i tilpasningen er at sikre, at tilstedeværelsen af negative vægte ikke fejlagtigt fører til en "optimal" delsekvens, der i realiteten har en lavere samlet vægt end mulige alternativer uden disse negative vægte.

Korrekt tilpasning for negative vægte

1. Initialiser en hjælpetabel W, der indeholder den kumulative vægt af delsekvensen op til hvert punkt. For hver celle i W, beregnes vægten som summen af vægten for den aktuelle celle plus vægten af den foregående celle, hvis denne sum er positiv. Hvis summen er negativ, sættes cellens vægt til 0, hvilket effektivt ignorerer negative bidrag til den samlede delsekvensvægt.

2. Anvend den dynamiske programmeringsalgoritme som normalt, men brug vægtene fra W til at bestemme den optimale delsekvens. Dette sikrer, at delsekvenser, der inkluderer elementer med negative vægte, kun vælges, hvis de bidrager positivt til den samlede delsekvensvægt.

3. Den endelige beregning af den optimale delsekvens's vægt kræver ikke yderligere justering, da behandlingen af negative vægte allerede er indarbejdet i opbygningen af W.

Denne tilgang ændrer ikke den overordnede struktur af den dynamiske programmeringsalgoritme og bevarer dermed den oprindelige algoritmes asymptotiske køretid. Den sikrer også, at den fundne delsekvens faktisk repræsenterer den højeste mulige samlede vægt under hensyntagen til muligheden for negative vægte i inputvektorerne.

5 Binomialkoefficienter

Lad n og k være heltal, $n \geq k \geq 0$. Binomialkoefficienten $\binom{n}{k}$ repræsenterer antallet af måder at vælge k forskellige elementer fra en mængde med n elementer. Binomialkoefficienten kan udtrykkes som:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} = \prod_{i=0}^{k-1} \left(\frac{n-i}{k-i} \right)$$

a) Ligheder for Binomialkoefficienter

Vi ønsker at bevise følgende egenskaber for binomialkoefficienten $\binom{n}{k}$:

$$\binom{n}{k} = \begin{cases} 1 & \text{hvis } k = 0 \text{ eller } k = n, \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{hvis } 0 < k < n. \end{cases}$$

Bevis

Basis tilfælde: For $k = 0$ og $k = n$, gælder det at:

- $\binom{n}{0} = 1$, da der kun er én måde at vælge nul elementer fra en hvilken som helst mængde, hvilket er at vælge ingen elementer.
- $\binom{n}{n} = 1$, da der kun er én måde at vælge alle n elementer fra en mængde af n elementer, hvilket er at tage hele sættet.

Induktivt skridt: Antag $0 < k < n$. Overvej et specifikt element, a, i mængden. Antallet af k-kombinationer fra en n-element mængde kan opdeles i to dele:

1. Kombinationer, der inkluderer a: At vælge $k-1$ yderligere elementer fra de resterende $n-1$ elementer, eksklusive a, svarer til $\binom{n-1}{k-1}$.

2. Kombinationer, der ikke inkluderer a: At vælge k elementer fra de resterende $n-1$ elementer, eksklusive a, svarer til $\binom{n-1}{k}$.

Summen af disse to muligheder udgør alle mulige måder at danne k-kombinationer fra n elementer, hvilket giver:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

Dette fuldender det induktive bevis for egenskaberne ved binomialkoefficienten $\binom{n}{k}$.

b) Pseudokode for Beregning af Binomialkoefficienter

Vi præsenterer pseudokode for en algoritme, der effektivt beregner binomialkoefficienten $\binom{n}{k}$ ved hjælp af dynamisk programmering. Algoritmen minimerer antallet af nødvendige operationer til højst nk additioner.

b) Pseudokode og køretidsanalyse

```
ALGORITME BeregnBinomialkoefficient(n, k)
1. Initialiser en tabel C med størrelsen (n+1) x (n+1) sat til 0
2. For i fra 0 til n gør:
     a. Sæt C[i][0] = 1 // Basis tilfældet, da C[n][0] altid er 1
     b. For j fra 1 til min(i, k) gør:
         i. Hvis j == i, sæt C[i][j] = 1
         ii. Ellers, sæt C[i][j] = C[i-1][j-1] + C[i-1][j]
3. Returnér C[n][k]
```

Analyse: Den indre løkke (2.b) udfører op til k additioner for hver iteration af den ydre løkke, som kører $n+1$ gange. Men i praksis løber j kun op til det mindste af i og k, hvilket begrænser antallet af faktiske additioner. Det samlede antal additioner er derfor summen af de første k naturlige tal, hvilket er $\frac{k(k+1)}{2}$, for hver i, indtil k, og derefter k additioner for hver i fra $k+1$ til n, hvilket giver en øvre grænse på nk. Dette viser, at det samlede antal additioner, der er nødvendige for at udfylde tabellen C, er højst nk, hvilket gør algoritmen effektiv med hensyn til antallet af operationer.